

Step 1 – Type of inputs

A csv/text file -

1. Class configuration –

Section, Subject, Teacher_ID, Duration, Frequency, IsLab

A, Maths, T1, 1, 4, 0

A, Physics, T2, 1, 5, 0

A, Physics_Lab, T2, 2, 2, 1

A, Chemistry, T3, 1, 4, 0

B, Maths, T1, 1, 4, 0

B, English, T4, 1, 5, 0

B, Chemistry_Lab, T3, 2, 2, 1

S, Advanced_AI, T5, 1, 3, 0

S, AI_Lab, T5, 2, 2, 1

2. Room configuration –

Room_ID, Capacity, Type

R1, 60, CLASSROOM

R2, 60, CLASSROOM

R3, 40, CLASSROOM

LAB1, 30, LAB

LAB2, 30, LAB

3. Section Configuration –

Section_ID, Strength, IsSpecial

A, 55, 0

B, 50, 0

S, 25, 1

4. Teacher Configuration –

Teacher_ID, Name, MaxHoursPerDay

T1, Dr. Smith, 6

T2, Prof. Jones, 5

T5, Dr. AI Expert, 4

5. Constraints –

const int MAX_DAYS = 5; // Mon to Fri

const int MAX_SLOTS = 8; // 9AM to 5PM (1 hour each)

const int MAX_ROOMS = 10;

const int MAX_TEACHERS = 20;

const int MAX_SECTIONS = 15;

const int MAX_SUBJECTS = 30;

const int LUNCH_SLOT = 4; // 1-2 PM (0-indexed: slot 4)

enum RoomType { CLASSROOM, LAB };

Step 2 – Converting CSV to Class Object

Create them on the basis of the above input provided.

The following classes might miss a few things or so from above input, just capture them on the basis of input.

The Logic:

1. Read requirements.csv line by line using `std::ifstream` and `std::stringstream`.
2. Explode the Requirements: When your code reads SecA, Math, T1, 1, 4 (Frequency is 4) it must run a loop 4 times to create four separate `ClassEvent` objects.
3. Store them all in a single master `std::vector<ClassEvent> allEvents;`

Core Data structures -

1. Room Information –

```
struct Room {  
    string roomID;    // "R1", "LAB1"  
    int capacity;     // 60, 30  
    RoomType type;    // CLASSROOM or LAB  
};  
  
Room rooms[MAX_ROOMS];  
int totalRooms;
```

2. Section Information –

```
struct Section {  
    string sectionID; // "A", "B", "S"  
    int strength;     // Number of students  
    bool isSpecial;   // Special section flag  
};  
  
Section sections[MAX_SECTIONS];  
int totalSections;
```

3. Teacher Information –

```
struct Teacher {  
    string teacherID;    // "T1", "T2"  
    string name;        // Optional: "Dr. Smith"  
    int max_hours_per_day  
};  
Teacher teachers[MAX_TEACHERS];
```

```
int totalTeachers;
```

4. Subject Information –

```
struct Subject {  
    string subjectID;    // "Maths", "Physics_Lab"  
    bool isLab;          // true if it's a lab  
};  
Subject subjects[MAX_SUBJECTS];  
int totalSubjects;
```

5. Boolean Flags (Constraints) –

```
bool teacherBusy[MAX_TEACHERS][MAX_DAYS][MAX_SLOTS];  
bool sectionBusy[MAX_SECTIONS][MAX_DAYS][MAX_SLOTS];  
bool roomBusy[MAX_ROOMS][MAX_DAYS][MAX_SLOTS];  
bool subjectOnDay[MAX_SECTIONS][MAX_DAYS][MAX_SUBJECTS];
```

Step 3 – Constraints and Master Table

teacherBusy[Teacher][Day][Slot] → Is this teacher occupied?

sectionBusy[Section][Day][Slot] → Is this section occupied?

roomBusy[Room][Day][Slot] → Is this room occupied?

These are **lookup tables** for fast constraint checking, not the final output.

timetable[Room][Day][Slot] = (Section, Subject, Teacher) , (Section, Subject, Teacher...

Each cell will contain the details of all the sections who have their classes at that time.

The Rooms dimension allows parallel classes.

After creation of the Master table, loop through each day and for every slot (the Lunch slot) Set the slot as true for no classes.

Apply a sorting algorithm on the class vector (allEvents) and sort them on the basis of whether they are Labs or not (Along with their duration) . **We will place the labs first. MCV Heuristic.**

Step 4 – Solving the time table

The solver is a recursive function that takes your sorted allEvents vector and tries to place them on the 3D chessboard.

The Flow:

1. Take the first ClassEvent from the vector.
2. Loop through Days (0-4), Slots (0-7), and Rooms.
3. At every coordinate, call a function named isSafe().
4. If isSafe() returns true, mark the 3D arrays as true (place the class).
5. Call the solver again for the next event in the vector.

Step 5 Backtracking algorithm –

5. Backtracking and Advanced Algorithms in Action

Constraint Validator (isSafe): This function checks the 3D arrays in $O(1)$ time. It ensures no teacher, section, or room is double-booked. It also checks the subjectOnDay array to ensure a section doesn't get Math twice in one day.

- **Depth-First Search (DFS) & Backtracking:** If the algorithm places 40 classes successfully but the 41st class has zero open slots, it has hit a dead end. It will **backtrack**—undoing the 40th placement (changing the 3D arrays back to false), moving it to a different slot, and trying again.
- **Forward Checking (Pruning):** Right after placing a class, the system scans the *remaining* unscheduled classes. If it sees that placing Math on Monday just completely ruined all options for Physics, it instantly backtracks without going deeper.

Member 1: ME

- Responsibility: Will write the recursive DFS solver that attempts to place the events on the 3D chessboard.
- Document Tasks: Step 4 (Solving the time table) and the DFS backtracking logic from Step 5.

Member 2: (File I/O & Structs)

- Responsibility: Converting the raw CSV text files into usable C++ objects. Will write the parser that reads requirements.csv and "explodes" a frequency of 4 into four distinct ClassEvent objects.
- Document Tasks: Step 1 (CSV inputs) and Step 2 (Struct generation for Room, Section, Teacher, Subject).

Member 3: (Constraints & State)

- Responsibility: Managing the rules. Will build the isSafe() validation function, which checks the 3D boolean lookup tables (teacherBusy, sectionBusy, roomBusy, subjectOnDay) in O(1) time to prevent double-booking. They are also responsible for pre-blocking the lunch slots.
- Document Tasks: Step 3 (Master Table initialization) and the isSafe() logic from Step 5.

Member 4: (Heuristics + Output)

- Responsibility: Making the algorithm fast and the output readable. Will implement the MCV heuristic by sorting the allEvents vector to place labs first. They will also write the Forward Checking (Pruning) logic and the final function that prints the 3D array to the console.
- Document Tasks: Sorting logic in Step 3, Pruning from Step 5, and the final visual output.

The flow of creation –

Member 2 (High Priority) – Immediately define the C++ structures in a shared common.h file so that everyone else can use it. Then begin writing the CSV parser.

Member 3 – Initialize the global 3D Boolean arrays.

Phase 2 –

Member 3 -

Write the isSafe() function. This is the referee for the whole project.

Member 2-

Finish the CSV parser and successfully populate the vector<ClassEvent> allEvents.

Member 4 –

Write the sorting function to order the populated vector (Labs first).

Phase 3 –

Member 1-

Write the DFS Backtrack loop.

Member 4 –

Start drafting the UI/Output formatter that will eventually read the finished timetable.

Phase 4 –

Member 4 –

The basic backtracker will likely be too slow. Inject the Forward Checking (Pruning) logic right after a class is placed to catch dead ends instantly. Finish the console output.